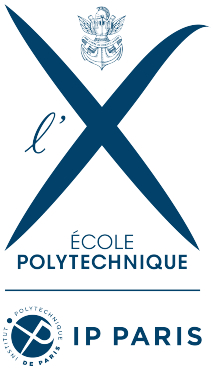

École Polytechnique

Concurrent Programming Final Project

Disease Spread Simulation

Student:

Manon Vu Huu

Professor:

Eric Goubault

Academic year 2025/2026

Contents

1 Introduction **3**

2 Model explanation **3**

3 Agents **4**

4 Sequential implementation **5**

5 Concurrent implementation **5**

6 Comparison of the two implementations **6**

7 Correctness and output **8**

8 Limitations & Future Work **8**

9 Conclusion **9**

10 References **9**

A Appendix **10**
A.1 Animated simulation 10

1 Introduction

Agent-based models can be used to study systems composed of many individual actors whose simple local behaviors can collectively produce complex global dynamics. Such models appear in many fields, including economics, epidemiology, traffic modeling, and social sciences. However, simulating them can quickly become computationally expensive, since the state and interactions of a large number of agents must be updated at every step of the simulation. This makes agent-based models a relevant target for parallelization.

In this project, we study an agent-based model of disease spread inspired by the paper *Modelling disease outbreaks in realistic urban social networks* by Eubank et al. (2004). In this work, the authors model disease propagation through a dynamic network of people and locations, where infections occur when susceptible and infectious individuals are present in the same place at the same time. Their approach shows how individual mobility patterns can strongly influence the global evolution of an epidemic.

The objective of this project is not to reproduce the full complexity of the original model, but rather to implement a simplified version that preserves its central idea: agents move between locations during the day, and the disease spreads through co-location. We first implement a sequential simulation, then develop a concurrent version in which the infection computation is parallelized across locations. The goal is to compare both implementations in terms of correctness, structure, and performance, and to analyze whether this type of agent-based model benefits from parallel computation.

2 Model explanation

This project is based on the paper *Modelling disease outbreaks in realistic urban social networks*, published in *Nature* by Eubank et al. in 2004. In this paper, the authors simulate the spread of smallpox in the city of Portland by using an agent-based model built from realistic mobility data. The central idea is that disease transmission does not happen through a uniformly mixed population, but through contacts between people who are present in the same locations at the same time.
They represent the model as a bipartite graph composed of two types of vertices: people and locations. An edge between a person and a location means that the person visited that location during a given time interval. Disease transmission is then computed from co-location events: if an infectious person and a susceptible person are in the same location at the same time for long enough, the susceptible person may become infected. This is the key modelling idea that we keep.

Our implementation is a simplified version. Instead of using real census and mobility data, we generate an artificial population of N agents moving routinely between houses and public locations over 50 days. The environment is composed of:

$$\text{number of houses} = \left\lceil \frac{25}{100} N \right\rceil \qquad \text{number of locations} = \left\lceil \frac{1}{10} N \right\rceil$$

Each agent has a daily schedule from 6:00 to 00:00. In the morning and evening, agents are mostly at their home, and during the day they visit a number of public locations. This creates repeated contacts between agents, similarly to the people-locations structure described in the paper, but in a simpler and fully generated setting.
Each agent is always in one of the following disease states:

$$\text{healthy} \to \text{exposed} \to \text{infectious} \to \begin{cases} \text{recovered} \\ \text{dead} \end{cases}$$

A healthy agent is susceptible to infection, and is less susceptible when vaccinated. Indeed, the US Center for Disease Control and Prevention states that the vaccine prevents smallpox for 95% of the vaccinated. An exposed agent is infected but not infectious yet. After an incubation and prodromal period, the agent becomes infectious and can transmit the disease to others. At the end of the infectious period, the agent either recovers and goes back to being healthy, or dies. Below is a detailed breakdown of the formulas used.

The transmission probability is derived from the smallpox parameters described by Eubank et al., who state that 95% of susceptible agents exposed for 3 hours to a person at minimum infectivity become infected. We convert this into an hourly transmission probability p by solving:

$$1 - (1 - p)^3 = 0.95 \implies p = 1 - 0.05^{1/3} \approx 0.632$$

So one hour of exposure to an infectious agent corresponds to a base infection probability of approximately 63.2%. For a contact lasting h hours, this generalizes to:

$$P(\text{infection} \mid h) = 1 - (1 - p)^h$$

During the 4-day infectious period, infectivity decreases exponentially with d, the number of days since the agent became infectious:

$$p_{\text{eff}}(d) = p \cdot e^{-\lambda d}$$

where λ is a decay parameter controlling the rate of decrease. The full infection probability for a contact of h hours on day d of infectiousness is therefore:

$$P(\text{infection} \mid h,\, d) = 1 - \big(1 - p_{\text{eff}}(d)\big)^h$$

Infected agents pass through sequential compartments before becoming infectious:

Incubation	7–17 days	$\mathcal{N}(\mu = 12,\ \sigma = 2)$, truncated
Prodromal	3–5 days	Uniform
Infectious	4 days	Fixed

After the infectious period, outcomes are determined stochastically: 70% of agents recover and 30% die, with death occurring 10-16 days after rash onset. The modifiable number of days of the simulation is set to 30 because given the periods in the table above, this allows for the observation of at the very least 1 longest cycle.

Overall, the model keeps the main structure of Eubank et al.'s approach: agents move through a set of locations, contacts occur through co-location, and infection depends on both disease state and exposure duration. However, our version is intentionally simpler. Its purpose is not to reproduce a real epidemic in Portland, but to provide a meaningful agent-based simulation that can be implemented sequentially and then parallelized.

3 Agents

Before running the simulation, we generate a parametrizable and randomized database of agents. This is done using a Python script, which creates an `agents.csv` file containing the population. The important parameters are the number of agents N, the proportion of vaccinated agents, and the number of initially infected agents. The schedules are generated randomly but follow a routine. Each agent is assigned a home, stays home before 9:00 and after 18:00, and visits between one and three public locations during the day. Initially infected agents are selected randomly among the population.

Column	Description
id	Unique integer identifier of the agent
home	Home location of the agent
loc_06:00 to loc_23:00	Agent location at each hourly time slot
disease_state	Current disease state: healthy, exposed, infectious, recovered, dead
vaccinated	Whether the agent is vaccinated
days_exposed	Number of days since the agent became exposed
days_infectious	Number of days since the agent became infectious
incubation_days	Duration of the incubation period for this agent
prodromal_days	Duration of the prodromal period for this agent
will_die	Whether the agent will eventually die from the infection
days_until_death	Number of days before death if applicable
ever_infected	Whether the agent has ever been infected
times_infected	Number of times the agent has been infected
ever_recovered	Whether the agent has ever recovered
times_recovered	Number of times the agent has recovered
last_day_infection_probability	Infection probability computed during the last simulated day
max_daily_infection_probability	Maximum daily infection probability observed for the agent
cumulative_infection_probability	Cumulative probability of infection over the simulation

4 Sequential implementation

The sequential implementation is written in C++ and uses the generated agents.csv file as input. At the beginning of the simulation, the program loads all agents, their disease states, vaccination statuses, and daily schedules. It then builds a location-based representation of the day: for each location, the program stores the agents who visit it and the hours at which they are present.

The simulation then proceeds day by day. For each day, the program first computes possible new exposures. This is done by iterating sequentially over all locations. At each location, the program separates infectious agents from healthy ones. Then, for every possible pair, it computes the number of overlapping hours spent in that location. If the contact duration is at least the minimum contact threshold (usually 1 hour), the infection probability is computed using the formula described in the previous section. Since a healthy agent may encounter several infectious agents in different locations during the same day, we combine the risks multiplicatively by accumulating the probability of not being infected. At the end of the exposure computation, a random draw determines whether each agent actually becomes exposed.

After the exposure step, the disease state of every agent is updated. Newly infected agents become exposed, exposed agents progress toward the infectious state according to their incubation and prodromal durations, and infectious agents either recover or die after the infectious period. The program writes a final summary containing aggregate statistics. Daily snapshot generation is available but is commented out for time optimization.

5 Concurrent implementation

The concurrent implementation keeps the same model and input as the sequential version, but parallelizes the most expensive part of the simulation: the computation of new exposures. The main idea is to use the model's structure. Since infections are computed locally inside each location, different locations can be processed concurrently during the same day. In addition, we also parallelize the disease progression step by splitting the agent vector into chunks. Each worker updates a disjoint interval of agents, so this step does not require locks on the agents themselves.

(The simulation itself cannot be fully parallelized over time, because the state of day $n + 1$ depends on the result of day n. So the loop over days stays sequential.)
At the beginning of each day, the program builds the same location-based representation as in the sequential version. Each task corresponds to one location. Worker threads then take locations from a shared task queue and process them independently. For each location, a worker identifies the infectious agents and the healthy non-vaccinated agents present there, computes their overlapping contact hours, and calculates the resulting infection probabilities.
The concurrent version uses several standard C++ concurrency tools:

Tool	**Role in the implementation**
`std::thread`	Creates worker threads
`join()`	Waits for workers to finish
`std::queue<size_t>`	Stores location indices as tasks
`std::mutex`	Protects the task queue
`std::lock_guard`	Locks/unlocks the queue mutex safely
`std::atomic<double>`	Stores shared no-infection probabilities
`std::atomic<size_t>`	Counts processed locations

The task distribution is implemented using a shared queue of location indices. Each worker repeatedly pops a location from the queue, processes it, and then takes another one until the queue is empty. A location containing many infectious and healthy agents is more expensive to process than a mostly empty location. Using a shared task queue therefore gives a simple form of dynamic load balancing between threads.

The main difficulty is that the same healthy agent may visit several locations during the same day. Therefore, several worker threads may try to update the infection risk of the same agent at the same time. To avoid data races, the implementation stores the daily probability of *not* being infected for each agent in a shared vector <atomic<double». Since several locations may contribute to the same agent's infection probability, workers update this vector using an atomic multiply. This avoids data races without using one mutex per agent.

Then, as in the sequential version, a random draw determines whether each agent actually becomes exposed. This step is done only after all threads have joined, which ensures that all location contributions have been computed before the disease states are updated.

Overall locations are produced as tasks, worker threads consume these tasks, and shared results are protected using mutexes. This approach preserves the logic of the sequential implementation while allowing the most computationally expensive part of each simulated day to run in parallel.

6 Comparison of the two implementations

To evaluate the sequential and concurrent implementations, we ran benchmarks for $N = 10000, 50000, 100000,$ and 200000 agents. For the concurrent version, we tested 1, 2, 3, 4, 6, 8 worker threads. For each configuration, the simulation was repeated 3 times, and we report the average runtime.

The benchmarks were run on the following hardware:
System: Darwin, Architecture: arm64, Processor: arm, Nb logical CPU cores: 8

The results show that the concurrent implementation improves performance more clearly than in the previous version, especially for larger populations. For $N = 10000$, the runtime is very small, so the overhead of creating and coordinating threads is comparable to the sequential work. As a result, the best speedup is about

Agents	Threads	Runtime (s)	Speedup
10000	Sequential	0.090	1.000
10000	1	0.108	0.833
10000	2	0.084	1.072
10000	3	0.087	1.027
10000	4	0.087	1.035
10000	6	0.093	0.964
10000	8	0.102	0.882
50000	Sequential	0.495	1.000
50000	1	0.582	0.849
50000	2	0.436	1.135
50000	3	0.441	1.121
50000	4	0.422	1.171
50000	6	0.468	1.056
50000	8	0.496	0.997
100000	Sequential	1.654	1.000
100000	1	1.835	0.901
100000	2	1.220	1.356
100000	3	1.053	1.570
100000	4	1.039	1.591
100000	6	1.078	1.535
100000	8	1.146	1.443
200000	Sequential	4.299	1.000
200000	1	4.741	0.907
200000	2	3.042	1.413
200000	3	2.566	1.676
200000	4	2.289	1.878
200000	6	2.494	1.724
200000	8	2.588	1.661

1.07, with 2 threads, and using 1, 6, or 8 threads is slower than the sequential implementation. For larger populations, the benefit of parallelism becomes much more visible. For $N = 50000$, the best result is obtained with 4 threads, reducing the runtime from 0.495 seconds to 0.422 seconds, corresponding to a speedup of about 1.17. For $N = 100000$, 4 threads again give the best result, reducing the runtime from 1.654 seconds to 1.039 seconds, which gives a speedup of about 1.59. The largest improvement is observed for $N = 200000$, where 4 threads reduce the runtime from 4.299 seconds to 2.289 seconds, corresponding to a speedup of about 1.88.

Another important observation is that using more threads does not always lead to better performance. For example, for $N = 200000$ and $N = 100000$, 4 threads give the best runtime. This suggests that after a certain point, the additional parallelism is outweighed by overheads such as thread scheduling, task queue synchronization, atomic updates, and memory contention. The 1 thread concurrent version is also slower than the sequential implementation for every tested population size, which is expected because it performs the same work while paying the cost of concurrent infrastructure.

Overall, the results show that the model benefits from parallel computation when the population is large enough. The best results are obtained with 4 worker threads, rather than with the maximum number of available logical cores. This indicates that the implementation exposes useful parallelism, but that synchronization and memory-access costs still limit scalability.

7 Correctness and output

To check the correctness of the simulation output, we generate daily snapshots of the population state and plotted the number of agents in each disease state over time. These counts are then represented as a line plot, with one curve per disease state.

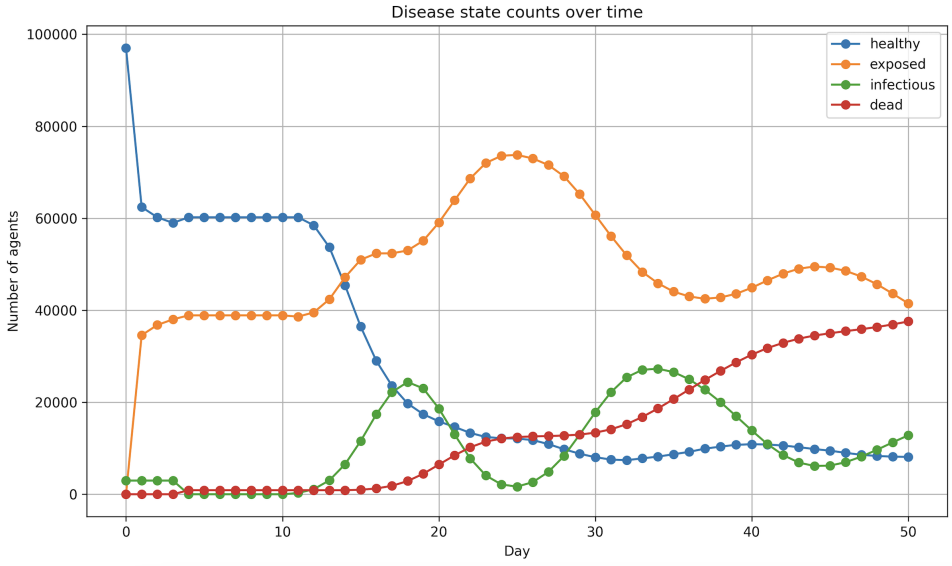

Figure 1: 50 Day simulation for N=aaaaa

At the beginning of the simulation, most agents are healthy, with a small number of infectious agents introduced initially. The number of healthy agents then decreases rapidly as infections spread through location contacts. At the same time, the number of exposed agents increases strongly, which is consistent with the incubation phase of the model. The infectious curve appears later than the exposed curve, which is also expected. This delay reflects the incubation and prodromal periods. The plot then shows waves of infectious agents, followed by increases in dead agents. The death curve is increasing because death is an absorbing state: once an agent is dead, it does not move to another state. The healthy curve decreases sharply at first, then stabilizes at a lower level, with small later variations caused by agents recovering and returning to the healthy state. The exposed and infectious curves also rise and fall over time, showing that agents move through the disease states in the intended order.

Overall, this visualization supports the correctness of the simulation output. It shows that the generated CSV snapshots are coherent with the model rules. It also confirms that the simulation produces interpretable epidemic behavior.

8 Limitations & Future Work

The first obvious limitation is that although the implementation captures the main idea of a location-based agent model, it remains much simpler than the model described by Eubank et al.: the population and mobility patterns are randomly generated rather than based on real census / traffic / land-use data, agents follow simple daily routines, and locations are not differentiated by type / capacity / social function. As a result, the simulated contact network is only a far approximation of a real urban social network.

The epidemiological model is also simplified: agents do not change their behavior when infected. In a more realistic model, infectious agents could stay home, be quarantined, or reduce their number of contacts. The model could also include different age groups, household structures, workplace structures, or heterogeneous susceptibility.

From a concurrent programming perspective, the main limitation is that the simulation cannot be fully parallelized over time. The current implementation parallelizes the 2 main computations inside each day: the location based exposure computation and the disease state update. For future work, we could use thread-local infection probability arrays and merge them at the end of each day, thereby reducing contention on shared atomic variables. Another direction would be to implement the exposure computation on a GPU, since many contact computations are independent.

9 Conclusion

In this project, we implemented a simplified agent-based model of disease spread inspired by Eubank et al.'s work on realistic urban social networks. We first developed a sequential version, then a concurrent version in which both location-based exposure computation and disease progression are parallelized.
The benchmarks show that the concurrent implementation successfully improves performance for larger populations, with the best results obtained using 4 worker threads. The speedup is not linear in the number of threads because of synchronization costs, atomic updates, memory contention, and the fact that the loop over days remains inherently sequential. We thus show both the potential and the limits of parallelizing agent-based simulations: the model contains natural parallelism, but efficient implementation requires careful control of shared data and coordination overhead.

10 References

[1] S. Eubank, H. Guclu, V. S. A. Kumar, M. V. Marathe, A. Srinivasan, Z. Toroczkai, and N. Wang. *Modelling disease outbreaks in realistic urban social networks*. Nature, 429, 180–184, 2004.

[2] M. Matsumoto and T. Nishimura. *Mersenne Twister: A 623-dimensionally equidistributed uniform pseudo-random number generator*. ACM Transactions on Modeling and Computer Simulation, 8(1), 3–30, 1998. DOI: https://doi.org/10.1145/272991.272995

[3] Centers for Disease Control and Prevention. *Smallpox Vaccine*. CDC, October 23, 2024. https://www.cdc.gov/smallpox/vaccines/index.html

[4] My GitHub repository. https://github.com/j3am6s/concurrent_project

A Appendix

A.1 Animated simulation

We made a visual animation of the simulation. The Python script displays locations as squares and agents as small circles inside them. The color of each circle represents the disease state of the agent, and a green border indicates vaccination. The animation progresses through both days and hours, using the agents' schedules to place them in the correct location at each time step.
This representation is less useful for large populations, because the display quickly becomes crowded. However, it provides an intuitive view of how agents move between locations, how infected agents can come into contact with healthy agents, and how the epidemic evolves spatially over time. And it was very fun to make.

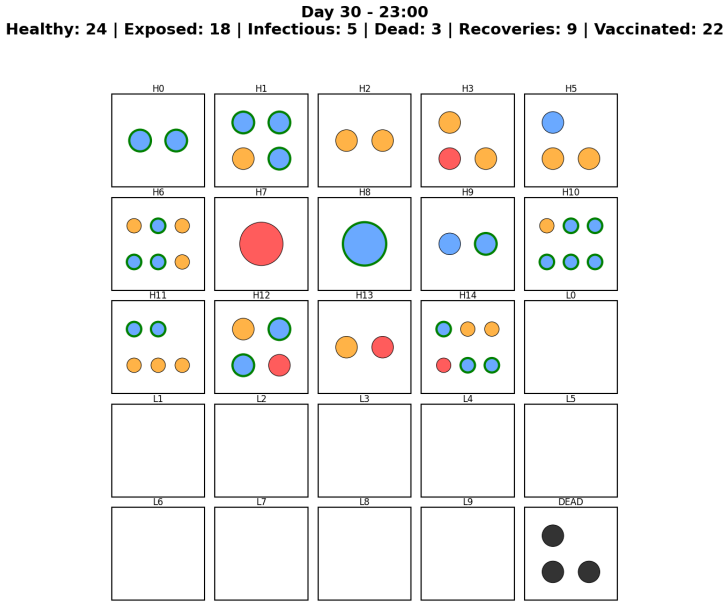

Figure 2: Last frame of a 30 day animation for N=50